MetaMask VedaAdapter Audit Report

Prepared by Cyfrin

Version 2.0

Lead Auditors

Kage

Immeas

April 20, 2026

Contents

1 About Cyfrin **2**

2 Disclaimer **2**

3 Risk Classification **2**

4 Protocol Summary **2**

5 Audit Scope **2**

6 Executive Summary **2**

7 Findings **4**

7.1 Low Risk . . . 4

7.1.1 Front-runner can override withdrawal token and slippage parameters in permissionless `VedaAdapter` functions . . . 4

7.1.2 Public batch execution is economically griefable because a single pre-consumed delegation reverts the entire batch . . . 5

7.1.3 `VedaAdapter` acts as a shared `msg.sender` for all Teller withdrawals, allowing any entity authorized for the `TellerWithMultiAssetSupport::deposit` to lock the adapter and block all user withdrawals . . . 7

7.2 Informational . . . 9

7.2.1 Deployment script instructs using plaintext private key via CLI argument . . . 9

7.2.2 Deployment script uses hardcoded zero-address placeholders instead of environment variables 9

7.3 Gas Optimization . . . 10

7.3.1 Use `calldata` instead of `memory` for external function array parameters . . . 10

7.3.2 Eliminate single-use `encodedTransfer_` local variable . . . 10

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

DeleGator is an account abstraction protocol previously audited by Cyfrin which is designed to enable flexible permission delegation across different types of Ethereum accounts.

5 Audit Scope

The audit scope was limited to PR166:

```
src/helpers/VedaAdapter.sol
```

6 Executive Summary

Over the course of 3 days, the Cyfrin team conducted an audit on the VedaAdapter code provided by MetaMask. The findings consist of 3 Low severity issues with the remainder being informational and gas optimizations.

Summary

Project Name	VedaAdapter
Repository	delegation-framework
Commit	d3338bb23b3a...
Fix Commit	aba8aa550ee3...
Audit Timeline	Apr 6th - Apr 8th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	3
Informational	2
Gas Optimizations	2
Total Issues	7

Summary of Findings

[L-1] Front-runner can override withdrawal token and slippage parameters in permissionless `VedaAdapter` functions	Partially Resolved
[L-2] Public batch execution is economically griefable because a single pre-consumed delegation reverts the entire batch	Acknowledged
[L-3] `VedaAdapter` acts as a shared `msg.sender` for all Teller withdrawals, allowing any entity authorized for the `TellerWithMultiAssetSupport::deposit` to lock the adapter and block all user withdrawals	Acknowledged
[I-1] Deployment script instructs using plaintext private key via CLI argument	Acknowledged
[I-2] Deployment script uses hardcoded zero-address placeholders instead of environment variables	Resolved
[G-1] Use `calldata` instead of `memory` for external function array parameters	Resolved
[G-2] Eliminate single-use `encodedTransfer_` local variable	Resolved

7 Findings

7.1 Low Risk

7.1.1 Front-runner can override withdrawal token and slippage parameters in permissionless `VedaAdapter` functions

Description: `VedaAdapter::withdrawByDelegation` and `VedaAdapter::depositByDelegation` are callable by anyone. The delegation framework's cryptographic signatures and caveats enforce the token being pulled from the delegator and the transfer amount, but do NOT enforce:

1. The withdrawal output token (`_token` parameter in `withdrawByDelegation`)
2. The minimum shares received (`_minimumMint` in `depositByDelegation`)
3. The minimum assets received (`_minimumAssets` in `withdrawByDelegation`)

These parameters are passed as function arguments by the caller and are not part of any signed caveat. A front-runner who observes a pending transaction can extract the delegation chain from the transaction calldata and submit their own transaction with altered parameters. The delegation's `ERC20TransferAmountEnforcer` running total is exhausted by the front-runner's transaction, consuming the single-use delegation.

Impact: The delegator receives fair-value assets in an unintended denomination (griefing, not fund theft). Slippage protection is bypassed, though the admin-controlled accountant rate limits practical slippage. The single-use delegation is consumed; the delegator must recreate the delegation chain to retry. The attacker pays gas but gains nothing financially. Practical likelihood is low on Arbitrum due to the centralized sequencer's FIFO ordering.

Proof of Concept:

1. Delegator creates a delegation chain for withdrawing 100 vault shares, intending to receive USDC with `_minimumAssets = 990_000000` (~1% slippage tolerance)
2. Front-runner observes the pending `withdrawByDelegation(delegations, USDC, 990_000000)` transaction
3. Front-runner extracts the `delegations` array from calldata and submits `withdrawByDelegation(delegations, WETH, 0)`
4. The delegation framework validates the chain and transfers 100 shares to the adapter (enforced by caveats)
5. The Teller's `withdraw(WETH, 100, 0, rootDelegator)` burns shares and sends WETH (not USDC) to the delegator at the accountant's fair-value rate, with zero slippage protection
6. The delegation's `ERC20TransferAmountEnforcer` running total is exhausted; the original transaction reverts

The same pattern applies to deposits: a front-runner can set `_minimumMint = 0`, removing the delegator's slippage protection.

Recommended Mitigation: Include the `_token`, `_minimumMint`, and `_minimumAssets` parameters as enforced fields within the delegation caveats (e.g., via an `AllowedCalldataEnforcer` or a custom enforcer), so these values are cryptographically bound to the delegator's intent and cannot be overridden by the caller.

MetaMask: Acknowledged. In our design, the slippage-related parameters (_minimumMint and _minimumAssets) primarily serve as sanity checks rather than core safety guarantees. While a front-runner can modify these values, this does not change the underlying economic value of the deposit or withdrawal, as the actual amount transferred is constrained by the delegation framework and vault logic. Regarding the withdrawal token, the VedaVault strictly limits the set of supported tokens. Any attempt to specify an unsupported token will revert, preventing loss of funds even if a front-runner attempts to alter this parameter. That said, we recognize that reducing the surface for parameter manipulation is desirable. To improve both security and gas efficiency, we are:

- Moving the deposit token configuration into the constructor so it is no longer a user-supplied parameter in withdrawal calls in commit `aba8aa5`
- Continuing to rely on a private mempool for transaction submission, which significantly mitigates mempool-based front-running scenarios

Cyfrin: Commit aba8aa5 verified. `depositToken` is now an immutable address provided in the constructor, no longer a user supplied parameter reducing the risk for manipulation.

7.1.2 Public batch execution is economically griefable because a single pre-consumed delegation reverts the entire batch

Description: `VedaAdapter::depositByDelegationBatch` and `VedaAdapter::withdrawByDelegationBatch` process streams sequentially and revert the full transaction if any one stream fails:

```
// VedaAdapter.sol:213-227
function depositByDelegationBatch(DepositParams[] memory _depositStreams) external {
    uint256 streamsLength_ = _depositStreams.length;
    if (streamsLength_ == 0) revert InvalidBatchLength();

    address caller_ = msg.sender;
    for (uint256 i = 0; i < streamsLength_;) {
        DepositParams memory params_ = _depositStreams[i];
        // @audit - if this reverts for ANY stream, the entire batch reverts
        _executeDepositByDelegation(params_.delegations, params_.minimumMint, caller_);
        unchecked {
            ++i;
        }
    }

    emit BatchDepositExecuted(caller_, streamsLength_);
}
```

The same pattern exists in `VedaAdapter::withdrawByDelegationBatch`.

Since `VedaAdapter::depositByDelegation` and `VedaAdapter::withdrawByDelegation` are callable by anyone and delegation chains are visible in the public mempool, an attacker can:

1. Monitor the mempool for a pending batch transaction
2. Extract a single delegation chain from the batch calldata
3. Front-run with a single `depositByDelegation` (or `withdrawByDelegation`) call using that delegation
4. The front-run tx succeeds — the delegation is consumed (enforcer `spentMap` exhausted), and the user receives their shares/assets
5. The operator's batch tx arrives — all streams execute until the consumed delegation reverts, reverting the entire batch

This makes public batching economically fragile. A batch can be invalidated by paying for only one execution, while the operator bears the cost of the reverted aggregate call.

Impact: This is an operational denial-of-service / reliability issue against public batching.

- Attackers cannot steal user funds
- Attackers can repeatedly force reverted public batches at materially lower cost than the operator's aggregate execution
- Under sustained mempool griefing, batching may become uneconomic unless the operator switches to private orderflow or to a batch design that tolerates per-stream failure

The practical severity depends on whether the protocol expects these batch entrypoints to be usable on the public mempool. If yes, this issue meaningfully degrades that design goal.

Proof of Concept: Add the following test to `VedaLending.t.sol`:

```
    /// @notice Demonstrates batch griefing: attacker front-runs 1 of 3 streams, entire batch reverts.
    ///         1. Operator constructs 3 deposit streams for Alice
    ///         2. Attacker front-runs stream[2] with single depositByDelegation
    ///         3. Stream[2] consumed, operator's batch reverts entirely
```

```
///         4. Only 1 of 3 users served, operator gas wasted on full batch
function test_POC_batchGriefingViaSingleFrontRun() public {
    uint256 aliceUSDCBefore_ = USDC.balanceOf(address(users.alice.deleGator));

    VedaAdapter.DepositParams[] memory streams_ = new VedaAdapter.DepositParams[](3);
    Delegation[] memory frontRunTarget_;

    // Build stream 0 (200 USDC, salt 100) - scoped to free stack
    {
        Delegation memory d_ = _createTransferDelegationWithSalt(
            address(users.bob.deleGator), address(vedaAdapter), address(USDC), type(uint256).max,
            ↪  100
        );
        Delegation memory r_ =
            _createAdapterRedelegationWithSalt(EncoderLib._getDelegationHash(d_), address(USDC),
            ↪  200e6, 100);
        Delegation[] memory c_ = new Delegation[](2);
        c_[0] = r_;
        c_[1] = d_;
        streams_[0] = VedaAdapter.DepositParams({ delegations: c_, minimumMint: 0 });
    }

    // Build stream 1 (300 USDC, salt 101)
    {
        Delegation memory d_ = _createTransferDelegationWithSalt(
            address(users.bob.deleGator), address(vedaAdapter), address(USDC), type(uint256).max,
            ↪  101
        );
        Delegation memory r_ =
            _createAdapterRedelegationWithSalt(EncoderLib._getDelegationHash(d_), address(USDC),
            ↪  300e6, 101);
        Delegation[] memory c_ = new Delegation[](2);
        c_[0] = r_;
        c_[1] = d_;
        streams_[1] = VedaAdapter.DepositParams({ delegations: c_, minimumMint: 0 });
    }

    // Build stream 2 (500 USDC, salt 102) - attacker will front-run this one
    {
        Delegation memory d_ = _createTransferDelegationWithSalt(
            address(users.bob.deleGator), address(vedaAdapter), address(USDC), type(uint256).max,
            ↪  102
        );
        Delegation memory r_ =
            _createAdapterRedelegationWithSalt(EncoderLib._getDelegationHash(d_), address(USDC),
            ↪  500e6, 102);
        Delegation[] memory c_ = new Delegation[](2);
        c_[0] = r_;
        c_[1] = d_;
        streams_[2] = VedaAdapter.DepositParams({ delegations: c_, minimumMint: 0 });
        frontRunTarget_ = c_;
    }

    // 2: Attacker front-runs with stream 2's delegation only
    vm.prank(makeAddr("Attacker"));
    vedaAdapter.depositByDelegation(frontRunTarget_, 0);

    assertGt(BORING_VAULT.balanceOf(address(users.alice.deleGator)), 0, "Alice got shares from
    ↪  front-run");

    // 3: Operator submits full 3-stream batch - reverts on consumed stream 2
    vm.prank(address(users.bob.deleGator));
    vm.expectRevert("ERC20TransferAmountEnforcer:allowance-exceeded");
```

```
        vedaAdapter.depositByDelegationBatch(streams_);

        // 4: Only stream 2 (500 USDC) executed. Streams 0+1 NOT served.
        assertEq(
            USDC.balanceOf(address(users.alice.deleGator)),
            aliceUSDCBefore_ - 500e6,
            "Only front-run stream executed. Batch streams 0 and 1 NOT served"
        );
    }
```

Recommended Mitigation: If public batching is expected to remain viable, the protocol should adopt one of these approaches:

1. **Use private orderflow for batch submission** Submit batches through a private mempool so delegation streams are not exposed for mempool front-running before inclusion.
2. **Change batch semantics to tolerate per-stream failure** Redesign batching to isolate failures per stream, for example by using external self-calls with `try/catch` and emitting per-stream failure events.

Metamask: Acknowledged. We agree that, in a fully public mempool setting, the described behavior can create an economically griefable pattern for batch execution, since a single pre-consumed delegation can cause an otherwise valid batch to revert. In practice, our operational model mitigates this in two ways: 1. We use a private mempool / private orderflow for submitting batch transactions, which substantially reduces the feasibility of mempool-based front-running and griefing of this kind. 2. Our backend batch orchestration logic detects failed delegations within a batch. If a delegation has already been consumed or is otherwise invalid, our system automatically adjusts the batch (e.g., by removing or correcting failing items) before resubmitting, thereby limiting the operational impact and cost of such failures. Given these measures, we consider the residual risk to be operational rather than a direct safety issue.

7.1.3 `VedaAdapter` acts as a shared `msg.sender` for all Teller withdrawals, allowing any entity authorized for the `TellerWithMultiAssetSupport::deposit` to lock the adapter and block all user withdrawals

Description: The Veda Teller's `withdraw` function enforces a share lock check against `msg.sender`:

```
// TellerWithMultiAssetSupport.sol:558-568
function withdraw(ERC20 withdrawAsset, uint256 shareAmount, uint256 minimumAssets, address to)
    external
    virtual
    requiresAuth
    nonReentrant
    returns (uint256 assetsOut)
{
    beforeTransfer(msg.sender, address(0), msg.sender); // @audit - checks VedaAdapter's lock, not the
    ↪  user's
    assetsOut = _withdraw(withdrawAsset, shareAmount, minimumAssets, to);
    ...
}
```

The `beforeTransfer` hook reverts if the `from` address has a non-expired share lock:

```
// TellerWithMultiAssetSupport.sol:378-388
function beforeTransfer(address from, address to, address operator) public view virtual {
    ...
    if (beforeTransferData[from].shareUnlockTime > block.timestamp) { // @audit - from = VedaAdapter
        revert TellerWithMultiAssetSupport__SharesAreLocked();
    }
}
```

When `VedaAdapter` calls `teller.withdraw()`, `msg.sender` is the adapter contract itself — shared across all users. Meanwhile, the 5-arg `deposit()` sets the share lock on an arbitrary `to` address:

```
// TellerWithMultiAssetSupport.sol:482-490
```

```
function deposit(
    ERC20 depositAsset, uint256 depositAmount, uint256 minimumMint,
    address to, // @audit - can be set to VedaAdapter's address
    address referralAddress
) external payable virtual requiresAuth nonReentrant returns (uint256 shares) {
    shares = _publicDeposit(depositAsset, depositAmount, minimumMint, to, referralAddress);
}
```

Which ultimately sets the lock:

```
// TellerWithMultiAssetSupport.sol:664
beforeTransferData[user].shareUnlockTime = block.timestamp + currentShareLockPeriod; // @audit - user =
↪  VedaAdapter
```

If any entity authorized for the 5-arg `TellerWithMultiAssetSupport::deposit()` selector calls `teller.deposit(token, 1, 0, vedaAdapterAddress, address(0))`, the adapter's `shareUnlockTime` is set, and every subsequent `teller.withdraw()` called by the adapter reverts for all users until the lock expires.

The root cause is a design mismatch: the Teller's share lock was built for direct user interactions where the depositor and withdrawer are the same `msg.sender`. The `VedaAdapter` breaks this assumption by acting as a shared intermediary, turning a per-user lock into a global lock surface.

Impact: If any entity besides `VedaAdapter` is authorized for the 5-arg `deposit()` selector on the Teller (or if it is made public), an attacker can:

- Block **all** withdrawals through `VedaAdapter` for up to `shareLockPeriod`
- Repeat the 1 wei deposit before each lock expiry to maintain an **indefinite DoS** on all adapter withdrawals
- Cost per lock period: 1 wei of any allowed deposit asset

All users who deposited through the adapter are unable to withdraw their funds for the duration of the attack.

Recommended Mitigation: Ensure the 5-arg `TellerWithMultiAssetSupport::deposit()` selector is restricted exclusively to the `VedaAdapter` in the RolesAuthority configuration, with no other entity sharing the role. This is an operational mitigation, not a code fix.

Alternatively keep a low enough `shareLockPeriod`, `0` (disabled), or a couple of seconds to ensure flash deposit/withdrawals cannot be done.

Metamask: Acknowledged: VedaVault will limit to VedaAdapter for deposit call with `to` parameter.

7.2 Informational

7.2.1 Deployment script instructs using plaintext private key via CLI argument

Description: The deployment script `DeployVedaAdapter.s.sol` documents usage with `--private-key $PRIVATE_KEY`, instructing deployers to pass the private key as a plaintext CLI argument sourced from an environment variable. The `.env.example` file contains a `PRIVATE_KEY=` entry confirming this pattern. While `.gitignore` covers `*.env` files, the key is still exposed in shell history, process listings, and CI logs.

Recommended Mitigation: Migrate to Foundry's encrypted keystore:

```
cast wallet import deployer --interactive
forge script script/DeployVedaAdapter.s.sol --rpc-url <rpc_url> --account deployer --broadcast
```

Update the NatSpec in the deployment script to document this secure pattern. Remove `PRIVATE_KEY` from `.env.example`.

MetaMask: Noted.

7.2.2 Deployment script uses hardcoded zero-address placeholders instead of environment variables

Description: `DeployVedaAdapter.s.sol` hardcodes all four constructor parameters as `address(0)` constants (lines 19-22), requiring Solidity source code modification before each deployment. This contrasts with other scripts in the same repository that use `vm.envAddress`. The `VedaAdapter` constructor will revert if deployed with these zero addresses, but requiring source code modification increases the risk of deploying a stale version or invalidating CREATE2 address predictions.

Recommended Mitigation: Use `vm.envAddress` for constructor parameters, consistent with other deployment scripts in the repository:

```
address owner = vm.envAddress("OWNER");
address delegationManager = vm.envAddress("DELEGATION_MANAGER");
address boringVault = vm.envAddress("BORING_VAULT");
address vedaTeller = vm.envAddress("VEDA_TELLER");
```

MetaMask: Fixed in commit 57b5b88

Cyfrin: Verified.

7.3 Gas Optimization

7.3.1 Use `calldata` instead of `memory` for external function array parameters

Description: Four external functions accept complex array parameters as `memory` when they are only read, not modified. Using `calldata` avoids an expensive copy from calldata to memory on every call. The two internal helper functions they delegate to can also be changed to `calldata` since they only read the delegations array:

```
// src/helpers/VedaAdapter.sol
    function depositByDelegation(Delegation[] memory _delegations, ...) external {
    function depositByDelegationBatch(DepositParams[] memory _depositStreams) external {
    function withdrawByDelegation(Delegation[] memory _delegations, ...) external {
    function withdrawByDelegationBatch(WithdrawParams[] memory _withdrawStreams) external {
    function _executeDepositByDelegation(Delegation[] memory _delegations, ...) internal {
        Delegation[] memory _delegations,
```

Recommended Mitigation: Change `memory` to `calldata` on all six function signatures:

```
- function depositByDelegation(Delegation[] memory _delegations, uint256 _minimumMint) external {
+ function depositByDelegation(Delegation[] calldata _delegations, uint256 _minimumMint) external {
```

Apply the same change to all other affected signatures. Update the loop body in batch functions to use `calldata` references accordingly.

MetaMask: Fixed in commit `1f1182e`

Cyfrin: Verified.

7.3.2 Eliminate single-use `encodedTransfer_` local variable

Description: In both `VedaAdapter::_executeDepositByDelegation` and `VedaAdapter::_executeWithdrawBy-Delegation`, the `encodedTransfer_` variable is assigned once and consumed on the immediately following line. Inlining the expression removes the extra local and avoids a trivial memory allocation:

```
// src/helpers/VedaAdapter.sol
    bytes memory encodedTransfer_ = abi.encodeCall(IERC20.transfer, (address(this), amount_));
    executionCallDatas_[0] = ExecutionLib.encodeSingle(token_, 0, encodedTransfer_);

    bytes memory encodedTransfer_ = abi.encodeCall(IERC20.transfer, (address(this), shareAmount_));
    executionCallDatas_[0] = ExecutionLib.encodeSingle(boringVault, 0, encodedTransfer_);
```

Recommended Mitigation: Inline the `abi.encodeCall` expression directly:

```
- bytes memory encodedTransfer_ = abi.encodeCall(IERC20.transfer, (address(this), amount_));
- executionCallDatas_[0] = ExecutionLib.encodeSingle(token_, 0, encodedTransfer_);
+ executionCallDatas_[0] = ExecutionLib.encodeSingle(token_, 0, abi.encodeCall(IERC20.transfer,
↪  (address(this), amount_)));
```

Apply the same change to the withdraw path.

MetaMask: Fixed in commit `adb6c64`

Cyfrin: Verified.